

Feature Scaling — Normalization

■ What's Covered

1. Quick Recap — Why Scaling is Needed
2. What is Normalization (Min-Max Scaling)?
3. The Min-Max Formula — Deep Dive
4. Step-by-Step Worked Example
5. Custom Output Range — `feature_range` Parameter
6. MinMaxScaler in Scikit-learn
7. Effect on Distribution
8. Sensitivity to Outliers — The Big Weakness
9. When to Use Normalization vs Standardization
10. MinMaxScaler vs Other Scalers
11. Common Mistakes
12. Summary

1. Quick Recap — Why Feature Scaling is Needed

When a dataset has numerical columns that live on vastly different scales, algorithms that rely on distances, dot products, or gradient descent are silently biased towards the features with the largest numeric values. Feature Scaling solves this by bringing every column onto a comparable range before training.

Scaling Technique	Core Idea
Standardization (Day 24)	Shift to mean=0, spread to std=1. Range is unbounded. Best default.
Normalization (Day 25 — THIS)	Compress to a fixed bounded range such as [0, 1]. Range is always known.
RobustScaler	Uses median & IQR — outlier resistant.
MaxAbsScaler	Divides by max absolute value — preserves sparsity.

2. What is Normalization (Min-Max Scaling)?

Definition: Normalization (also called Min-Max Scaling) transforms every value in a feature so that the entire column is compressed into a fixed, bounded range — most commonly [0, 1]. The minimum value in the column maps to 0 and the maximum value maps to 1. Every other value is scaled proportionally in between.

After normalization, a value of 0 means 'this observation has the smallest value seen in training for this feature' and a value of 1 means 'this observation has the largest value seen'. Every other value sits proportionally between 0 and 1.

2.1 Intuitive Analogy

Imagine converting exam marks to a percentage. If the highest mark in the class is 80 out of 100, normalizing maps $80 \rightarrow 1.0$ (top), $0 \rightarrow 0.0$ (bottom), and every other score scales proportionally. Now you can instantly compare marks across subjects that had different maximum scores — that is exactly what Min-Max scaling does for ML features.

2.2 Key Characteristic — Bounded Output

- After Min-Max scaling the output range is always $[0, 1]$ by default.
- You always know the minimum possible scaled value (0) and the maximum possible scaled value (1).
- Unlike Standardization, there is no such guarantee — Z-scores can take any value, positive or negative.
- The custom range $[a, b]$ can also be specified — e.g., $[-1, 1]$ or $[0, 255]$ — using the `feature_range` parameter.

3. The Min-Max Formula — Full Breakdown

$$x' = (x - x_{\min}) \div (x_{\max} - x_{\min})$$

Symbol	Meaning
x	Original value of the data point for a given feature
$x_{\min}$	Minimum value of that feature in the TRAINING set
$x_{\max}$	Maximum value of that feature in the TRAINING set
x'	Scaled output value — always between 0 and 1 (default)
$x_{\max} - x_{\min}$	Called the 'range' of the feature in the training set

3.1 What the Two Parts Do

- $(x - x_{\min})$ — Shifting: subtracts the minimum so the smallest value becomes 0. All values become non-negative (assuming no negative raw values).
- $\div (x_{\max} - x_{\min})$ — Scaling: divides by the total range so the largest value becomes 1. All values are now fractions of the total range.

3.2 General Formula for Custom Range $[a, b]$

$$x' = a + [(x - x_{\min}) / (x_{\max} - x_{\min})] \times (b - a)$$

When $a=0$ and $b=1$ this reduces to the standard formula above. When $a=-1$ and $b=1$ the data is scaled to $[-1, 1]$, which is useful for neural networks with tanh activation functions.

3.3 Properties After Min-Max Scaling

- Minimum of scaled column = 0 (or a if custom range)
- Maximum of scaled column = 1 (or b if custom range)
- Mean is NOT fixed — it depends on data distribution
- Standard deviation is NOT fixed — depends on data spread
- Relative distances between values are PRESERVED (linear transform)
- Shape of the distribution is PRESERVED — same as standardization

Important: Min-Max scaling does NOT make the distribution normal. A right-skewed distribution stays right-skewed after normalization. Only the numeric range changes; the shape is untouched.

4. Step-by-Step Worked Example

Manually applying Min-Max scaling on two features — Age and Income — to see exactly what happens at each step.

4.1 Raw Data

Person	Age (years)	Income (■ lakh/yr)
A	20	3
B	30	6
C	40	9
D	50	12
E	60	15

4.2 Step 1 — Find Minimum and Maximum

```
# Find min and max for each feature
x_min_Age = 20, x_max_Age = 60
x_min_Income = 3, x_max_Income = 15

# Compute the range (denominator)
range_Age = 60 - 20 = 40
range_Income = 15 - 3 = 12
```

Python

4.3 Step 2 — Apply $x' = (x - x_{\min}) / (x_{\max} - x_{\min})$

```
# Person A: Age=20
x'_A_Age = (20 - 20) / 40 = 0 / 40 = 0.00

# Person B: Age=30
x'_B_Age = (30 - 20) / 40 = 10 / 40 = 0.25

# Person C: Age=40
x'_C_Age = (40 - 20) / 40 = 20 / 40 = 0.50

# Person D: Age=50
x'_D_Age = (50 - 20) / 40 = 30 / 40 = 0.75

# Person E: Age=60
x'_E_Age = (60 - 20) / 40 = 40 / 40 = 1.00
```

Python

4.4 Final Scaled Data

Person	Scaled Age x'	Scaled Income x'
A (min)	0.00	0.00
B	0.25	0.25
C	0.50	0.50
D	0.75	0.75
E (max)	1.00	1.00

Observation: Both Age (20–60) and Income (3–15) are now on the same [0, 1] scale. Notice the equal spacing — this data happened to be uniformly spaced, so scaled values are also equally spaced. For real data the spacing will reflect the original distribution.

4.5 What Happens to a New (Unseen) Value?

If a new person has Age = 35 at prediction time, we apply the SAME $x_{\min}$ and $x_{\max}$ from the training set — not from the new data.

```
# Applying training statistics to a new value
x_new_Age = 35
x'_new = (35 - 20) / 40 = 0.375

# If new value is OUTSIDE training range, e.g. Age = 70:
x'_out = (70 - 20) / 40 = 1.25 # > 1 !
# Warning: value outside [0,1] - model may extrapolate
```

Python

Critical: Values outside the training range produce scaled values outside [0, 1]. This is a known weakness of Min-Max scaling. Standardization does not have this boundary constraint.

5. Custom Output Range — feature_range Parameter

The default range is [0, 1] but you can scale to any [a, b] range by setting the feature_range parameter in MinMaxScaler.

Range	When to Use	Example Use-Case
[0, 1] (default)	Most common, neural networks (sigmoid output), image pixel fractions	Tabular ML, pixel values ÷ 255
[-1, 1]	Neural networks with tanh or symmetric activations; centered around zero	Deep learning, RNNs
[0, 255]	Inverse-scale normalized images back to pixel values	Image processing pipelines
[a, b] (custom)	When domain requires a specific output range	Sensor readings, business KPIs

Python

```
# Default range [0, 1]
from sklearn.preprocessing import MinMaxScaler

scaler_01 = MinMaxScaler(feature_range=(0, 1))

# Custom range [-1, 1] for neural networks with tanh
scaler_11 = MinMaxScaler(feature_range=(-1, 1))

# Custom range [0, 100] – e.g. to express as percentage
scaler_pct = MinMaxScaler(feature_range=(0, 100))
```

6. MinMaxScaler in Scikit-learn

Scikit-learn's MinMaxScaler learns the minimum and maximum of each feature from the training data and applies the transformation consistently to any new data using those stored values.

6.1 Core API

Python

```
# Import the scaler
from sklearn.preprocessing import MinMaxScaler

# Create instance – default range [0, 1]
scaler = MinMaxScaler()

# Fit on training data – learns x_min and x_max per feature
scaler.fit(X_train)

# Transform training data using learned statistics
X_train_scaled = scaler.transform(X_train)

# Transform test data using the SAME x_min and x_max from training
X_test_scaled = scaler.transform(X_test)

# Shortcut: fit and transform in one step (training only)
X_train_scaled = scaler.fit_transform(X_train)
```

6.2 Inspecting Learned Parameters

```
# Check what the scaler learned from training data
print('Feature minimums:', scaler.data_min_)
print('Feature maximums:', scaler.data_max_)
print('Feature ranges :', scaler.data_range_)
print('Scale factors :', scaler.scale_)
```

Python

6.3 Inverse Transform — Recover Original Values

```
# Reverse normalization to get back original scale
X_original = scaler.inverse_transform(X_train_scaled)

# Useful for interpreting model predictions in real-world units
```

Python

6.4 Correct Train-Test Workflow (No Leakage)

```
# Step 1: Split data FIRST – before any scaling
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Step 2: Fit on TRAINING data only – never on test
scaler = MinMaxScaler()
scaler.fit(X_train)

# Step 3: Transform both splits using training statistics
X_train_sc = scaler.transform(X_train)
X_test_sc = scaler.transform(X_test)

# Step 4: Train your model on the scaled training data
model.fit(X_train_sc, y_train)

# Step 5: Evaluate on the scaled test data
score = model.score(X_test_sc, y_test)
```

Python

6.5 Pipeline — Best Practice

Python

```
# Pipelines prevent data leakage automatically
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier

# Build pipeline: scaler → model
pipe = Pipeline([
    ('scaler', MinMaxScaler()),
    ('model', KNeighborsClassifier(n_neighbors=5))
])

# fit() auto-calls fit_transform on X_train, then model.fit
pipe.fit(X_train, y_train)

# predict() auto-calls transform on X_test before predicting
predictions = pipe.predict(X_test)
```

Using a Pipeline is the safest way to use MinMaxScaler. It guarantees that the scaler's `fit()` is called only on training data, even during cross-validation — where the split changes each fold.

7. Effect on Distribution

Min-Max scaling is a linear transformation — it stretches or compresses linearly. The effect on key distributional properties:

Property	Before Scaling	After Min-Max Scaling
Minimum value	Any (e.g., 20, 3)	Always 0 (or a)
Maximum value	Any (e.g., 60, 15)	Always 1 (or b)
Mean	Any value	Shifts — not fixed
Std Deviation	Any value	Changes — not fixed
Distribution shape	Normal/Skewed/Bimodal	Exactly same — UNCHANGED
Relative ordering	$A > B$ in raw data	$A > B$ in scaled — preserved
Unit	Original (kg, ■, yrs)	Dimensionless [0, 1]
Outlier position	Outlier exists	Outlier becomes value near 0 or 1; normal data compressed

8. Sensitivity to Outliers — The Big Weakness

This is the most critical weakness of Min-Max scaling. Because the formula uses $x_{\min}$ and $x_{\max}$ — the extreme ends of the data — a single extreme outlier will completely distort the scaling of ALL other values.

8.1 Worked Illustration

```
# Salary data with one extreme outlier
Salaries = [30000, 35000, 40000, 45000, 50000, 1000000]

# Min and max are dominated by the outlier
x_min = 30000, x_max = 1000000
range = 970000

# Normal salary of 40000 after scaling:
x'_40k = (40000 - 30000) / 970000 = 0.0103

# Normal salary of 50000 after scaling:
x'_50k = (50000 - 30000) / 970000 = 0.0206

# The outlier takes the maximum value:
x'_out = (1000000 - 30000) / 970000 = 1.0000

# Result: ALL normal salaries crushed into [0.0, 0.02]
# The model cannot distinguish between 30k and 50k salaries!
```

Python

8.2 Solutions

- **Option 1 — Clip:** Clip / Cap outliers before applying MinMaxScaler: set a maximum threshold (e.g., 99th percentile) and clip values above it.

Python

```
# Cap at 99th percentile before scaling
import numpy as np
upper = np.percentile(X_train['Salary'], 99)
X_train['Salary'] = X_train['Salary'].clip(upper=upper)
X_test['Salary'] = X_test['Salary'].clip(upper=upper)

# Now apply MinMaxScaler safely
scaler = MinMaxScaler()
X_train_sc = scaler.fit_transform(X_train)
```

- **Option 2 — Switch Scaler:** Use RobustScaler instead — it uses median and IQR, ignoring extreme values entirely.

Python

```
# RobustScaler: robust to outliers
from sklearn.preprocessing import RobustScaler

robust = RobustScaler()
X_train_sc = robust.fit_transform(X_train)
```

Rule: If your data has significant outliers, prefer RobustScaler or clip outliers first, then apply MinMaxScaler. If your data is clean and well-bounded, MinMaxScaler is ideal.

9. When to Use Normalization vs Standardization

Both are valid scaling techniques. Choosing between them depends on the data properties and the algorithm being used.

Factor	Choose Normalization (MinMaxScaler)	Choose Standardization (StandardScaler)
Output range needed?	Yes — you need a bounded [0,1] range	No preference — output range doesn't matter
Data distribution	Uniform or bounded distributions work well	Normal (Gaussian) distributions work perfectly
Outliers present?	Avoid — outliers heavily distort scaling	Less impacted, but still affected
Algorithm	Neural networks (sigmoid), image data, recommendation systems	SVM, PCA, Regularized Regression, most distance-based algorithms
Pixel data (images)	■ Natural choice: pixels ÷ 255 → [0,1]	Also used, but [0,1] more interpretable
Unknown data range?	Risky — future data may exceed training min/max	Safer — Z-scores accommodate unseen ranges

Practical Tip: When in doubt, try both and compare model performance with cross-validation. The difference is often small, but for neural networks MinMaxScaler (especially [0,1] or [-1,1]) is generally preferred over StandardScaler.

10. MinMaxScaler vs Other Scalers

Scaler	Formula	Key Properties
MinMaxScaler (Normalization)	$x' = (x - \min) / (\max - \min)$	Range [0,1]. Simple & interpretable. Sensitive to outliers.
StandardScaler (Standardization)	$z = (x - \mu) / \sigma$	Mean=0, Std=1. Unbounded. Less sensitive than MinMax.
RobustScaler	$x' = (x - \text{median}) / \text{IQR}$	Robust to outliers. Unbounded. Best for heavily skewed data.
MaxAbsScaler	$x' = x / \max(x)$	Range [-1,1]. Preserves zeros. Best for sparse data.
Normalizer (row-wise)	Scale each row to unit L2 norm	Per-sample, not per-feature. Used in NLP/text tasks.

11. Common Mistakes to Avoid

- ■ **Fitting on the full dataset before splitting:** Data leakage! Always split first, then fit scaler on training only.

- ■ **Using MinMaxScaler with significant outliers:** Outliers collapse all normal values to near 0. Use RobustScaler or clip first.
- ■ **Forgetting to transform test data:** Test data must be transformed with the SAME scaler fitted on training.
- ■ **Assuming output will always be in [0, 1]:** New data with values outside the training range will produce values outside [0, 1]. Handle or clip if needed.
- ■ **Scaling the target y for classification:** y is a class label — never scale it for classifiers.
- ■ **Applying to sparse data:** MinMaxScaler destroys sparsity by shifting. Use MaxAbsScaler instead.
- ■ **Not using a Pipeline in cross-validation:** Without a Pipeline, the scaler is fitted on full training data before each CV fold split — causing subtle data leakage.

12. Summary — Everything in One Place

Concept	Key Takeaway
What it does	Scales each feature to a fixed range — default [0, 1]
Formula	$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$
Custom range	$x' = a + [(x - \min) / (\max - \min)] \times (b - a)$ via <code>feature_range=(a,b)</code>
What is learned	<code>x_min</code> and <code>x_max</code> from the TRAINING set only
Output range	Always [0, 1] by default — bounded, unlike Standardization
Shape preserved?	Yes — linear transform, distribution shape unchanged
Mean / Std fixed?	No — unlike Standardization, mean and std are not fixed
Outlier sensitivity	HIGH — a single extreme value distorts all other scaled values
Solution for outliers	Use <code>RobustScaler</code> , or clip outliers before <code>MinMaxScaler</code>
sklearn class	<code>MinMaxScaler()</code> in <code>sklearn.preprocessing</code>
Fit on	Training data ONLY — never on test/validation
<code>data_min_</code>	Attribute storing the learned minimum per feature
<code>data_max_</code>	Attribute storing the learned maximum per feature
Best for	Neural networks, image data (pixels÷255), bounded/uniform distributions
Avoid for	Data with significant outliers, sparse data
vs <code>StandardScaler</code>	<code>MinMax</code> gives bounded output; <code>Standard</code> gives $\mu=0$, $\sigma=1$
Pipeline	Always wrap in sklearn Pipeline to prevent data leakage

Notes by: Amol Jagtap | amoljagtap3001@gmail.com | Day 25 — Feature Scaling: Normalization